

Lecture 2

Learning Modules

C-programming Basics / Integration and Process Concept

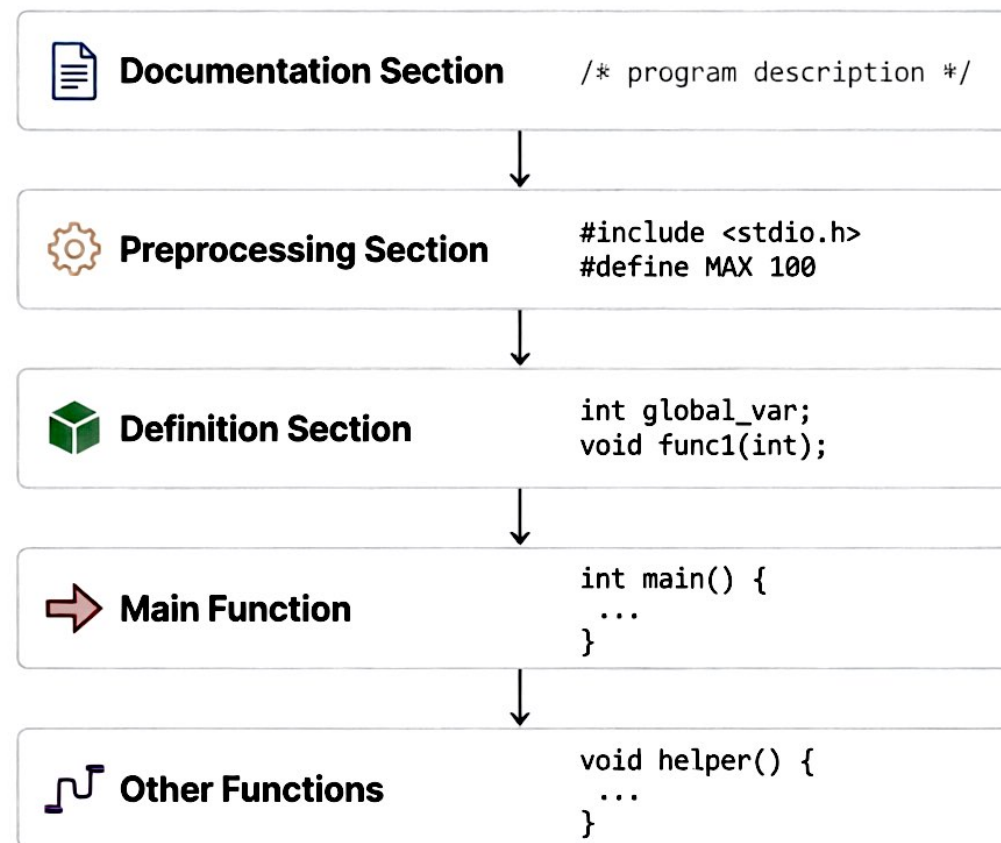
Learning Objectives :

1. The basic structure of C program.
2. The main() function and return type
3. Stages of compilation
4. Use of GCC flags
5. Understand distinction between a program and a process
6. How OS loads and executes a program?

1. C Programming Structure :

The structure of a C program is the initial stage. It determines the writing and execution of a C program. It begins with preprocessor directives, that contains libraries, then code blocks, statements and control flows, which are the main functions, is program logic and ends with return statement. It indicates successful program execution. Comments are also used. The code must be in order and maintain necessary flow.

C Program Structure Flow Diagram



Structure of a .c file :

/ Begin with comments about file contents */*

Insert #include statements and preprocessor definitions

Function prototypes and variable declarations

Define main() function

```
{  
    Function body  
}
```

Define other function

```
{  
    Function body  
}
```

GCC installation :

Use Linux based System for Practice.

Install build-essential package :

apt install build-essential

Check version :

gcc - -version

The main () Function :

The printf() function is used to display output. Parameters can be provided to get desired formatted output.

The first parameter “%s” inserted as string, the second parameter as “%i” is an integer.

```
int main() {  
    printf(" %s This is a formatted : %i", "Batch", 38);  
    return 0;  
}
```

The main () function after { parenthesis is the body where the main logic stays }.

Note : C is a compiled language, the source code must be converted to machine readable using a compiler. GNU Compiler Collection or gcc is widely available on many OS.

Working mechanism of C :

```
/* C program begins with comment */  
  
#include <stdio.h>  
  
int main() {  
    printf("This is Batch E38!:\n");  
    return 0;  
}
```

Source Code

Create a source file that contains readable C code.

```
MacBook-Air LABS % ./Structure
```

Run the file following way to see the result. ./ is path to the file.

```
MacBook-Air LABS % ls  
MacBook-Air LABS % gcc Structure.c -o Structure
```

Compile Code

run code through compiler to check errors.



Result

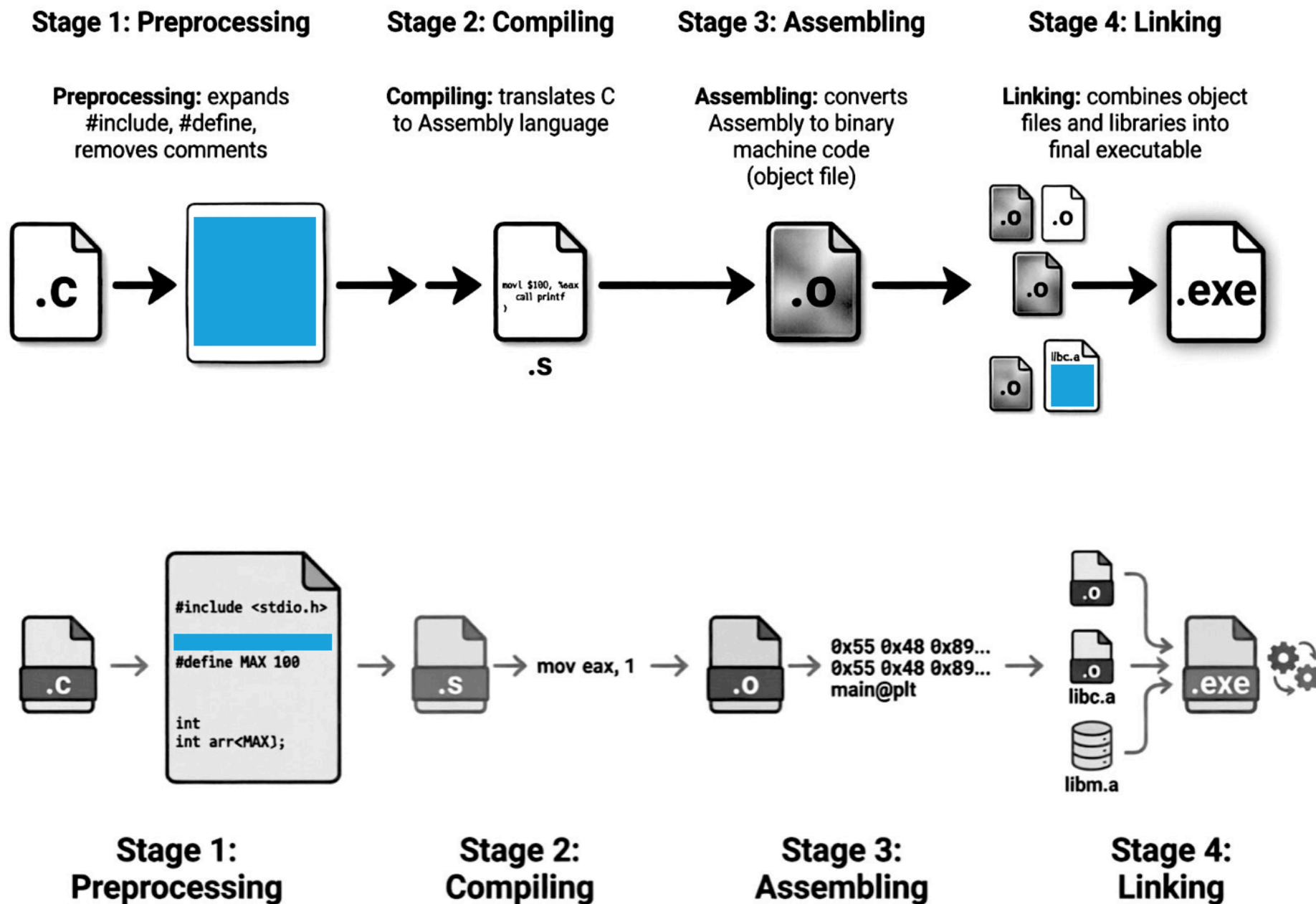
Compiler creates an executable file, it contains stream of 0s and 1s.

Run the file the output can be seen

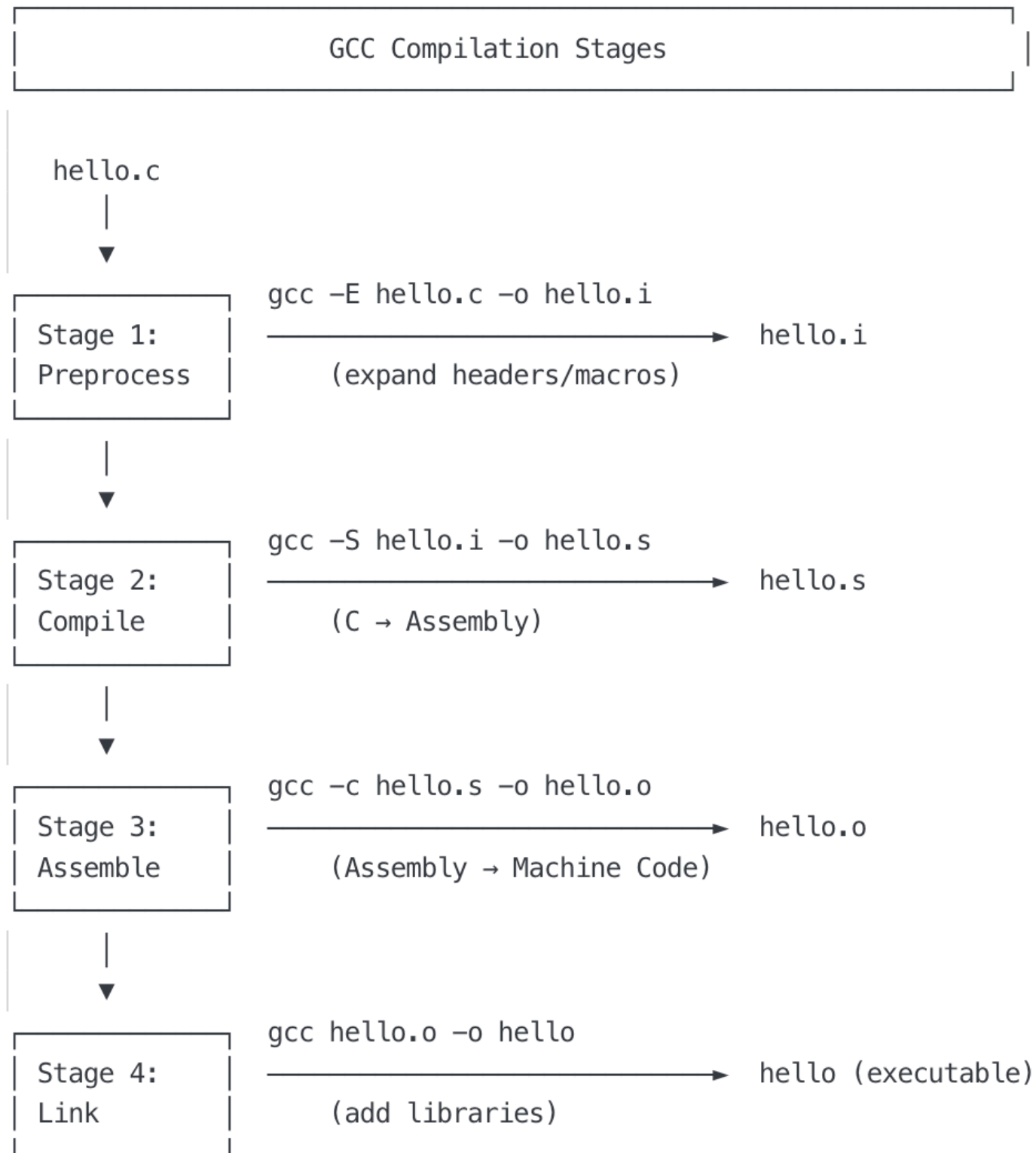
The program code will start from main () function. The main function has return type of int. The return value of the main () function determines if the program was successful. If it return other value, means error in the program.

C compilation :

C Compilation Flow



C compilation :



C Compilation Stages:

Stage	Command	Output File	Purpose
1. Preprocessing	gcc -E	.i	Expand headers & macros
2. Compiling	gcc -S	.s	Generate Assembly code
3. Assembling	gcc -c	.o	Create object file
4. Linking	gcc	.exe/a.out	Final executable

Task 4:

Try each section and take screenshot for submission

From : Preprocessing - Compile - Assemble - Link(automatically happens)

More Example :

Main () function, declare an integer variable *number*,

Compile and run the code,

Asks to enter a number, and it display the number.

```
/* C program begins with comment */
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int number;
```

```
    printf("Enter a number: ");
```

```
    scanf("%i", &number);
```

```
    printf("You entered: %i", number);
```

```
    return 0;
```

```
}
```

More Example :

```
/* C program begins with comment */

#include <stdio.h>

int main() {

    char user[10];
    printf("Enter user_name: ");

    scanf("%s", user);

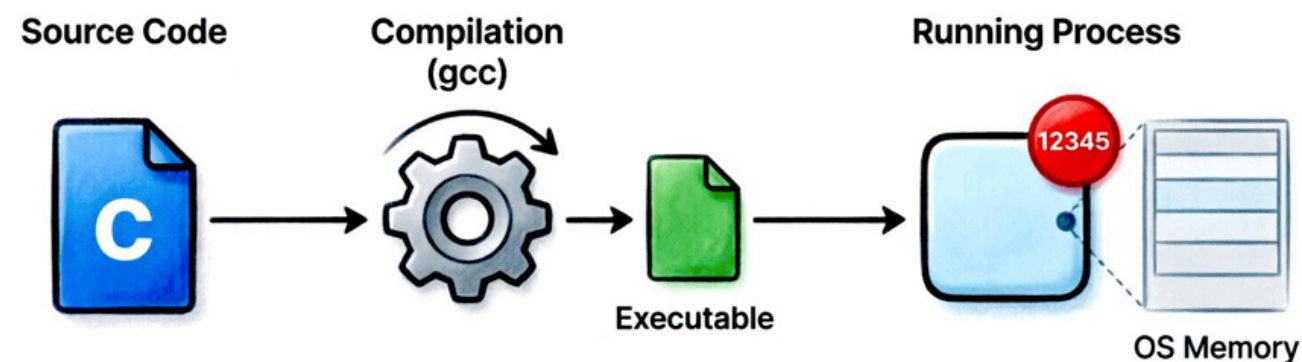
    printf("You entered username: %s", user);

    return 0;
}
```

Integration and Process Concept :

A computer program begins as **source code**, which is a human-readable set of instructions written in a programming language such as C. However, the CPU cannot directly understand this high-level language; it only executes instructions in **machine code**, which consists of binary values (0s and 1s). To bridge this gap, a **compiler** translates the source code into an **executable**, which contains machine-level instructions. This executable file is stored on disk and remains inactive until it is executed.

Program Lifecycle



When a user runs the executable, the **Operating System (OS)** plays a crucial role in transforming it into a **process**, which is a program in execution. The OS loads the executable into main memory (RAM), allocates necessary resources, and begins execution from the program's entry point (typically the `main( )` function). At this stage, the program becomes active and is managed by the OS throughout its lifecycle..

A file and an executable:

A key distinction exists between a **file** and a **process**. A file is a static entity stored on disk, while a process is dynamic, existing only while the program is being executed.

Human Readable

```
#include <stdio.h>

int main() {
    int number;
    printf("enter your number :");
    scanf("%d", &number);
    printf("your number is :%d\n", number);
    return 0;
}
```

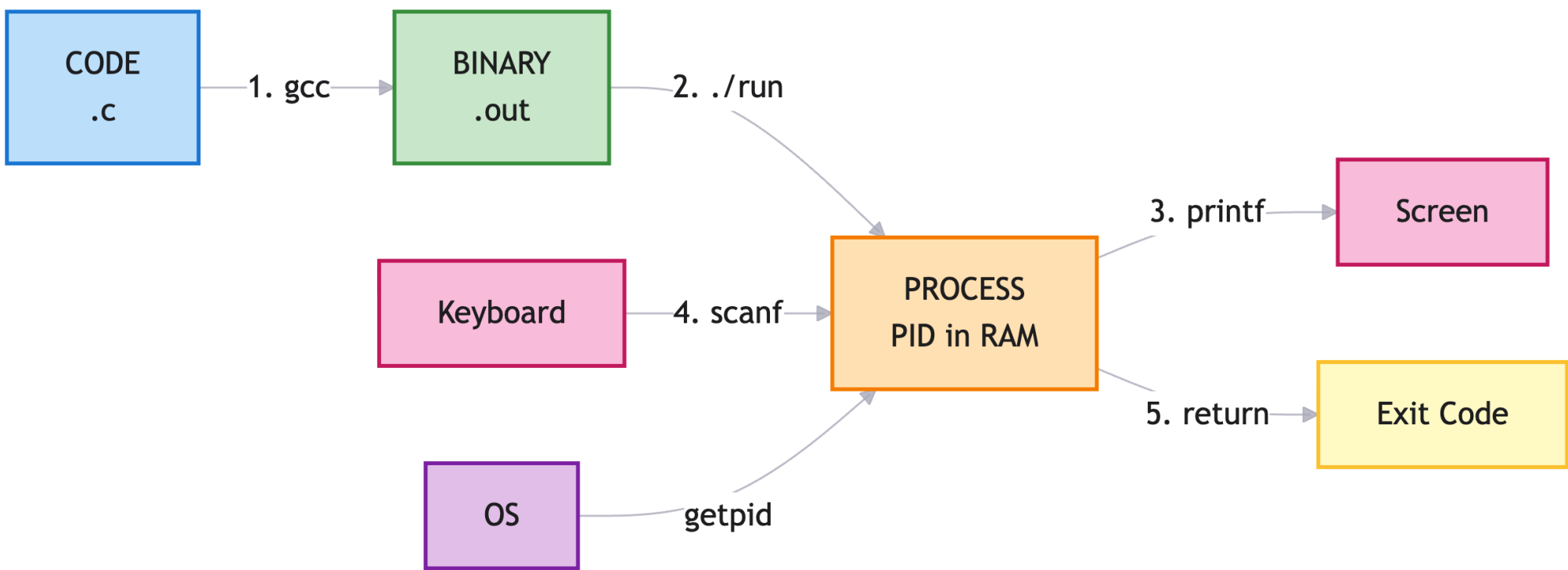
Machine Code

new.o: file format mach-o arm64

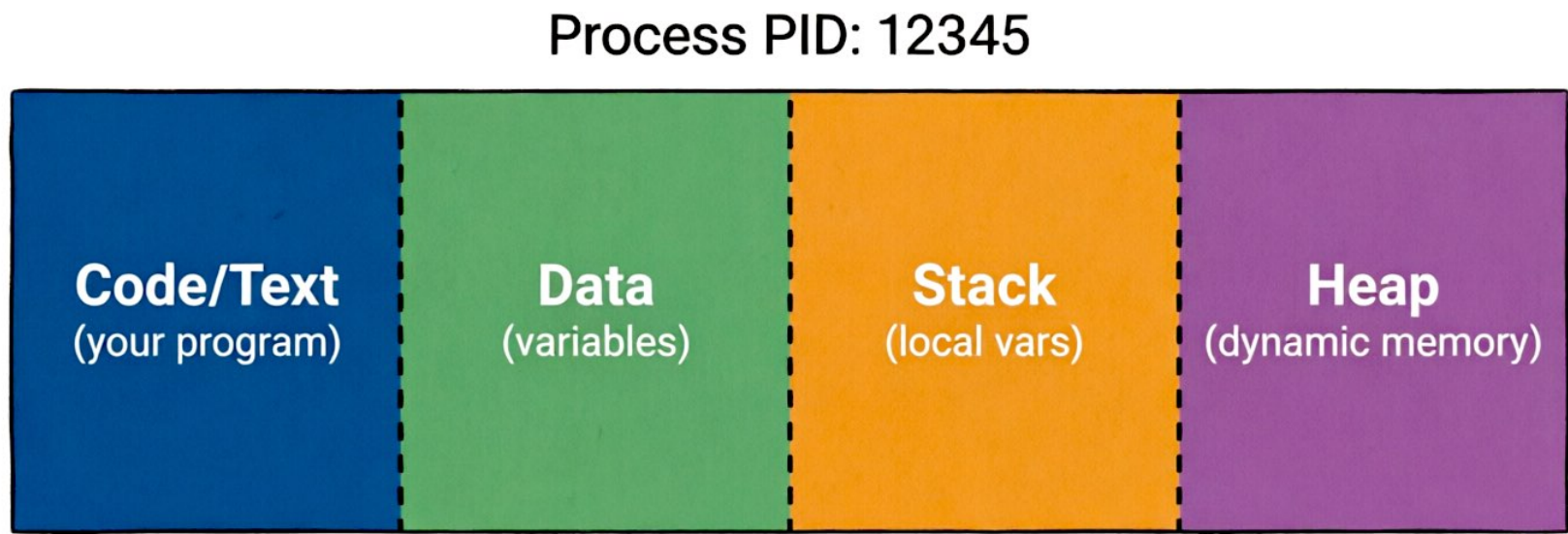
Disassembly of section __TEXT,__text:

```
0000000000000000 <tmp0>:
      0: ff 83 00 d1    sub     sp, sp, #32
      4: fd 7b 01 a9    stp     x29, x30, [sp, #16]
      8: fd 43 00 91    add     x29, sp, #16
     c: 08 00 80 52    mov     w8, #0
    10: e8 0b 00 b9    str     w8, [sp, #8]
    14: bf c3 1f b8    stur    wzr, [x29, #-4]
    18: 00 00 00 90    adrp    x0, 0x0 <tmp0+0x18>
    1c: 00 00 00 91    add     x0, x0, #0
    20: 00 00 00 94    bl      0x20 <tmp0+0x20>
    24: e0 0b 40 b9    ldr     w0, [sp, #8]
    28: fd 7b 41 a9    ldp     x29, x30, [sp, #16]
    2c: ff 83 00 91    add     sp, sp, #32
    30: c0 03 5f d6    ret
```

A Process Layout:



Memory Process Layout:

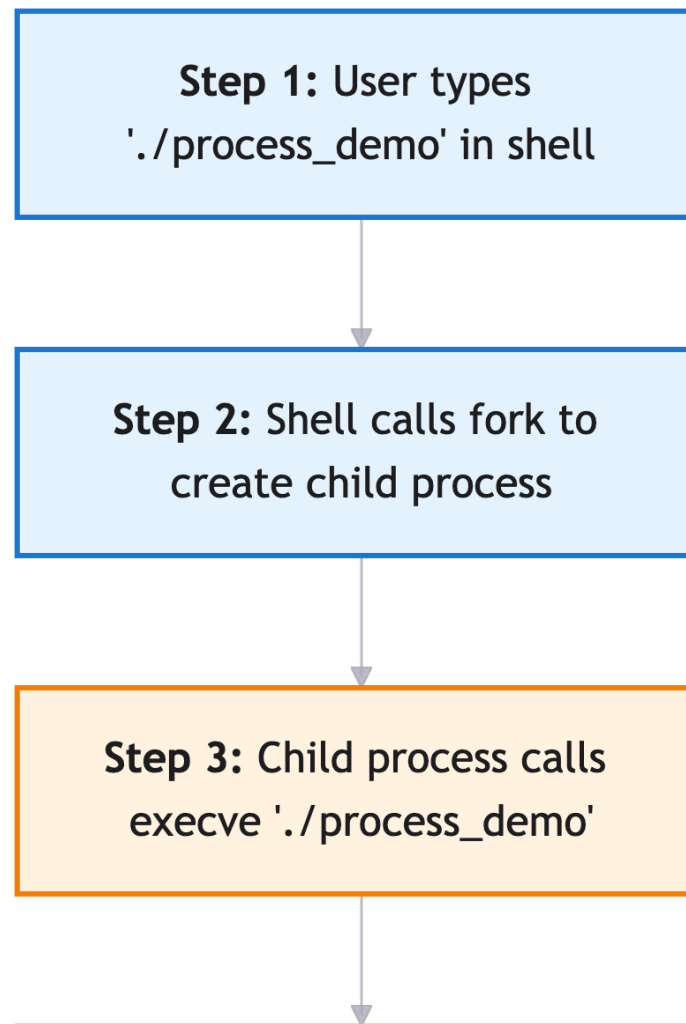


when a process is created, the OS organizes memory into different sections such as the **code segment** (instructions), **data segment** (global variables), and **stack** (local variables and function calls)

- 1. The OS is responsible for creating, scheduling, and terminating processes.
- 2. Each process is assigned a unique identifier called a **Process ID (PID)**, which allows the OS to track and manage it efficiently.
- 3. Processes may also have a **Parent Process ID (PPID)**, indicating which process created them.

Concept	Explanation
File	Static, stored on disk, never changes
Process	Dynamic, loaded in RAM, actively running
OS Role	The OS creates, manages, and destroys processes

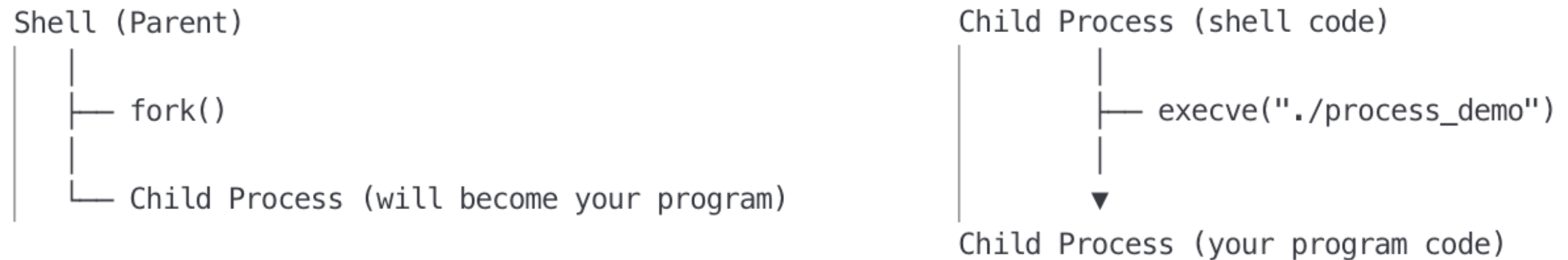
OS loads and Executes a program :



The child process replaces itself with your program using the **`execve()`** system call. This is where the executable file gets loaded.

— After the call of `execve()`, the OS replaces the shell code with the executable program.

OS loads and Executes a program :



The OS kernel takes control and prepares your program to run.

Step 4: OS Kernel Actions

- Opens executable file
 - Reads ELF header
- Allocates memory for process
- Loads code .text, data .data, BSS segments
 - Sets up stack with arguments & environment
- Jumps to entry point `_start`

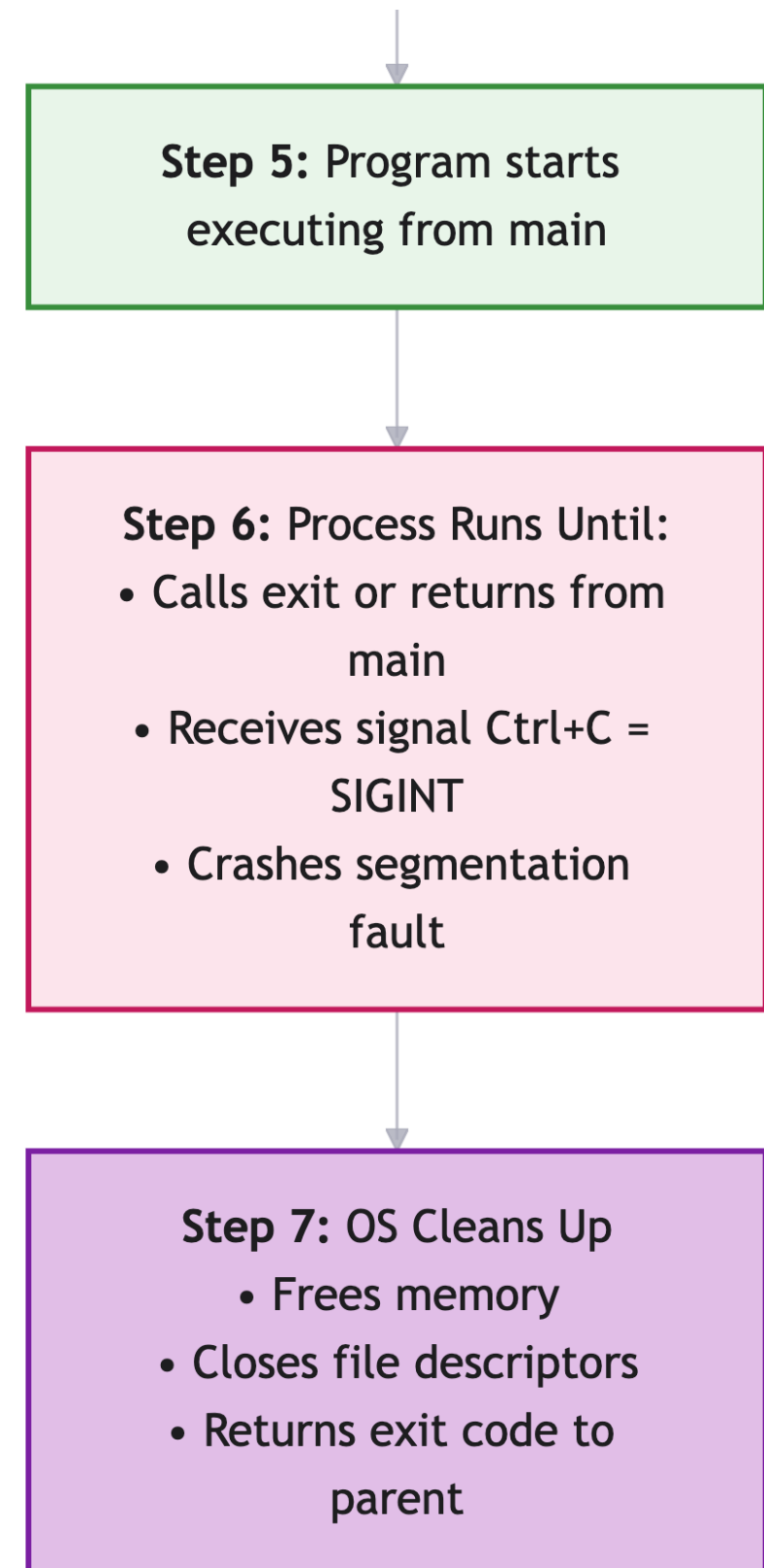
The kernel checks executables, assigns space in memory, moves in the code and date, then jumps to the entry point.

OS loads and Executes a program :

After the kernel sets up everything the execution begins.
Technically, it starts at **_start**, calls the **main ()** function.

— The compiler adds hidden code before it, the kernel jumps to **_start** it sets things up, then calls **main()**

Concept	Explanation
_start	Hidden entry point inserted by compiler (sets up environment)
main()	Your code's entry point (where you start writing)
Runtime	The period when your code is actively executing



OS loads and Executes a program :

The process continues executing until one of three things happen

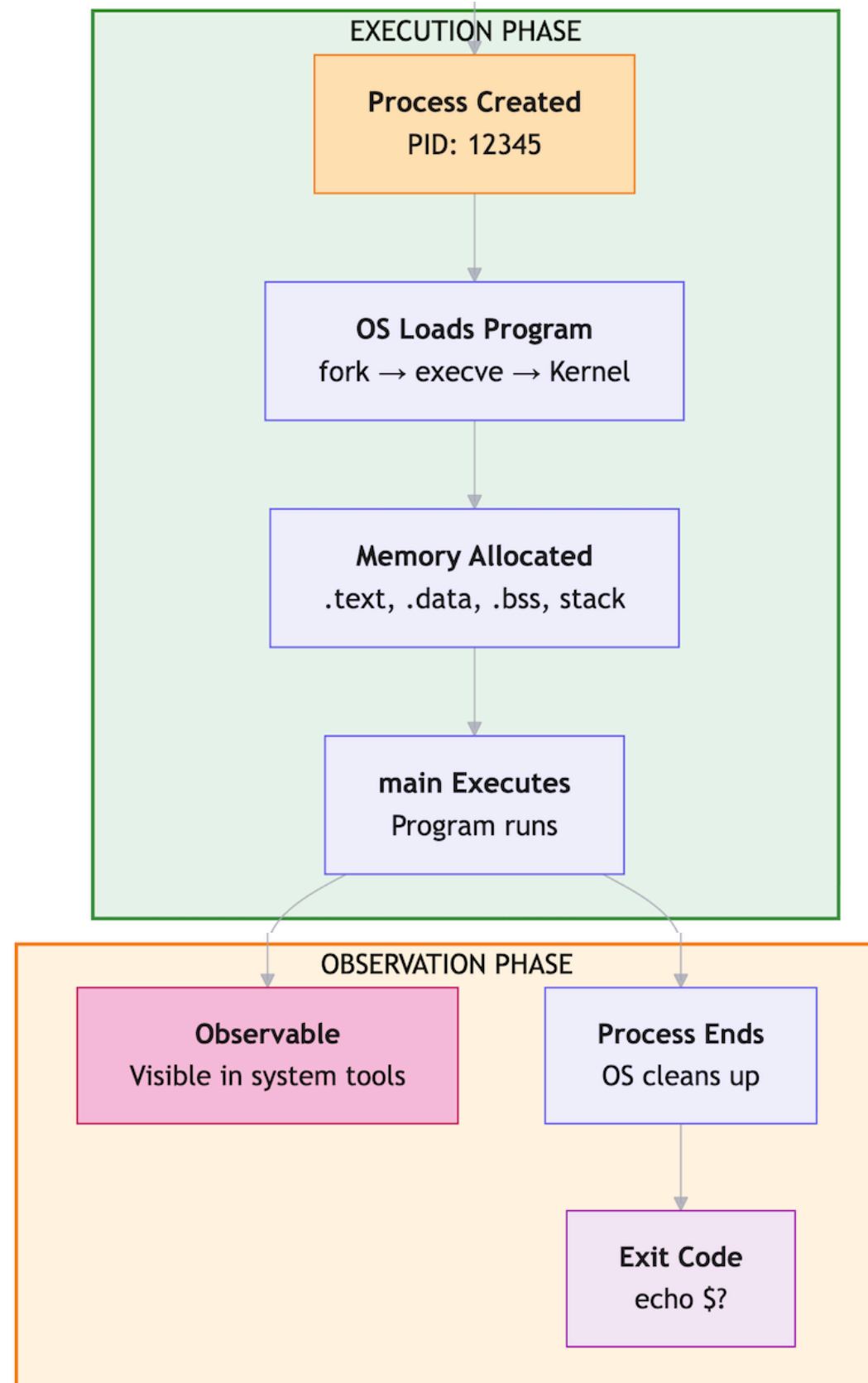
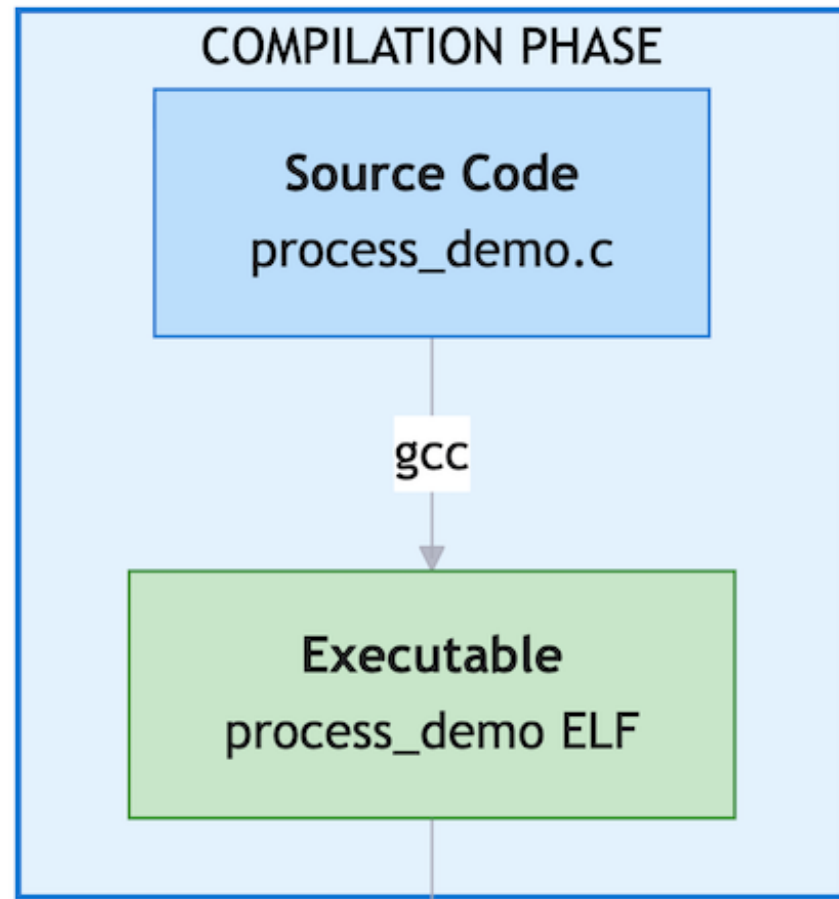
Termination Type	Cause	Example
Normal Exit	return from main() or exit() call	return 0;
Signal	External interrupt from OS or user	Ctrl+C (SIGINT)
Crash	Invalid operation (segfault, divide by zero)	Accessing invalid memory

The program runs line by line. It can end normally (return) , be stopped externally or crash with error.
— Each ending tells the OS something different.

When the process ends, the OS reclaims all resources.

Cleanup Action	Explanation
Frees memory	All RAM allocated to process is released
Closes file descriptors	Any open files are closed
Returns exit code	Status sent to parent process (shell)
Removes from process table	PID is freed for reuse

Overview :



LAB :

1. [Link to Lab Exercises on Routine](#)

Knowledge Test :

What is the difference between source code and an executable?

What command compiles a C program?

What is a process?

What does PID stand for?

How do you check the exit code of the last command?

What does return 0; mean in main()?

What system call creates a new process?

What system call replaces current process with a new program?

How do you see running processes in terminal?

What happens to memory when a process ends?

Where is your program stored before execution?

Where is your program stored during execution?

Why does the OS assign a PID to each process?

When does a process end?

Who assigns the PID?